PRODUCT MANAGEMENT SYSTEM

Frontend Architecture & System Documentation

TABLE OF CONTENTS

1. System Overview
2. Technology Stack
3. Core Architecture & Modules
4. Routing & Authentication Guard
5. State Management & API Layer
6. Reusable UI & Theme Components
7. Environment Configuration

1. SYSTEM OVERVIEW

The frontend is a responsive Single Page Application (SPA) built for a modern Product Management ERP system. It provides an intuitive interface for inventory tracking, catalog management, category organization, dynamic data entry forms, server-side tables, and visual dashboard analytics.

2. TECHNOLOGY STACK

- Framework & Language: React 19 + TypeScript
- Build Tool: Vite
- Styling: Tailwind CSS v4 + Sass (SCSS modules)
- State Management: Redux Toolkit + Redux Persist
- Routing: React Router DOM v7
- Forms & Validation: Formik + Yup schema validation
- Data Visualization: Recharts (Donut chart, horizontal/vertical bar charts)
- UI Primitives & Icons: Headless UI, React Icons (Feather & FontAwesome)
- Notifications: Sonner

3. CORE ARCHITECTURE & MODULES

The application is structured into domain-specific feature modules:

- Authentication (/login): Secure login screen validating credentials, saving JWT access and refresh tokens into persisted state.
- Dashboard (/admin/dashboard): Real-time metrics overview displaying top KPI summary cards (Total Products, Active, Categories, Low Stock, Out of Stock), product status donut chart, top 4 categories volume chart, and a responsive low-stock alerts table.
- Product Management (/admin/product): Comprehensive catalog management with server-side pagination, multi-column search and filters, image preview, in-row status dropdown updates, delete confirmation, and dynamic add/edit modal with Cloudinary upload.
- Category Management (/admin/category): Organization of product categories with paginated tables, search, CRUD operations, and feeds for product dropdown selects.

4. ROUTING & AUTHENTICATION GUARD

The routing system is organized hierarchically:

- Public Routes: Login and registration routes accessible only to unauthenticated visitors.
- Protected Admin Routes: Wrapped inside authentication guards verifying valid session tokens in Redux store. Unauthorized visits are automatically redirected to /login.
- Navigation Layout: Shared admin shell containing a responsive collapsible sidebar, top navigation bar, and breadcrumb path indicator.

5. STATE MANAGEMENT & API LAYER

- Redux Store: Manages global user authentication state, permissions, and profile details with localStorage persistence via Redux Persist.
- Centralized API Service (apiService): Unified wrapper for GET, POST, PUT, DELETE, and multipart/form-data requests.
- Anti-Duplicate Protection: Tracks active API request flags to prevent duplicate or accidental double-submissions.
- Token Refresh Interceptor: Automatically handles token expiration by passing the refresh token via headers to maintain seamless user sessions without logout.

6. REUSABLE UI & THEME COMPONENTS

The application relies on a shared, modular UI component library located in src/theme:

- Server Table (Table): Reusable data table supporting server-side pagination, sorting, column-level search filters, image rendering, in-row action menus, and inline status dropdowns.
- Smart Select (SelectBase): Custom searchable select with React Portal rendering, bound within viewport limits and featuring automatic upward flip detection when bottom space is constrained.
- Dynamic Modal (Modal): Accessible portal dialogs with smooth transitions for add/edit forms and confirmation alerts.
- Form Renderer (RenderField): JSON-driven field renderer translating schema definitions into Formik text inputs, textareas, selects, or image uploaders.

7. ENVIRONMENT CONFIGURATION

Required environment variables (.env):

- VITE_API_URL: Base URL for backend REST API endpoints (e.g., http://localhost:5000/api/v1/)
- VITE_IMAGE: Base URL or CDN path for static assets and image resolution